DATA 260 Homework 8

Akanksha Shukla

GitHub repository: https://github.com/akankshashuklaWork/data260-7838

Collaborators Sbnikitha and supriyaselvanganesan have access (checked with the GitHub API).

Personal configuration

Value	Result
SID4	7838
PORT_BASE	8638 (8000 + 7838 mod 900)
PREFIX	s7838
SEED	7838
VERIFY_SEED	267838
DOMAIN_ID	6 (7838 mod 8): Rental Housing Listings
Hardware	MacBook Pro (Apple M4 Pro, 48 GB unified memory)
Local model	qwen2:7b through Ollama
Tagged commit hash	hw8 tag not yet created (working commit c811a5eb3ec8)

Part 1 - Backend request/response over Kafka

Topics and message schemas

- Request topic: s7838_entity_requests. Response topic: s7838_entity_responses.
- Primary entity: RentalHousingListing, so CREATE_ENTITY is CREATE_LISTING and entity_id is listing_id. Topic names keep the literal word entity as written in the assignment.

```
# Request message (client/API -> s7838_entity_requests)
{"correlation_id": "uuid", "reply_to": "s7838_entity_responses",
 "operation": "CREATE_LISTING", "data": {"propertyTitle": "...", "propertyLocation": "...",
   "submitterEmail": "...", "propertyDescription": "(> 25 chars)",
   "propertyCategory": "Apartment|House|Condo|Townhouse", "termsAccepted": "true"}}
{"correlation_id": "uuid", "reply_to": "...", "operation": "GET_LISTING",
 "data": {"listing_id": "uuid"}}

# Response message (worker -> reply_to)
{"correlation_id": "same uuid", "success": true, "listing_id": "uuid", "message": "Listing created"}
{"correlation_id": "same uuid", "success": true, "listing": {"listing_id": "uuid", ...fields}}
{"correlation_id": "same uuid", "success": false, "error": "Listing not found"}
```

Complete correlation_id flow

- homework_demo.py creates one correlation_id and sends it in the HTTP request body.
- The FastAPI wrapper copies it into the Kafka request and keeps a pending future keyed by it.
- The worker copies it, unchanged, into the response (including every error response).
- FastAPI matches the response to the pending request by correlation_id and returns it; the demo checks that it still equals the original.

Client (FastAPI wrapper and Kafka client)

```
# code/hw8/app/kafka_rpc.py
    async def call(
        self, operation: Any, data: Any, correlation_id: str | None = None
```

```
    ) -> dict[str, Any]:
        # The correlation_id is created once; every later hop must preserve it.
        correlation_id = correlation_id or str(uuid.uuid4())
        request = {
            "correlation_id": correlation_id,
            "reply_to": CONFIG.response_topic,
            "operation": operation,
            "data": data,
        }
        future = asyncio.get_running_loop().create_future()
        self.pending[correlation_id] = future

        # Log before publishing: the worker can reply before send_and_wait even returns.
        log(
            "FASTAPI",
            f"sending request topic={CONFIG.request_topic} correlation_id={correlation_id} "
            f"operation={operation}",
        )
        log("FASTAPI", f"request message={json.dumps(request, separators=(',', ':'))}")
        await self.producer.send_and_wait(CONFIG.request_topic, to_bytes(request))
```

```
# code/hw8/app/main.py
@app.post("/listings")
async def listing_operation(request: ListingRequest, response: Response) -> dict[str, Any]:
    result = await rpc.call(request.operation, request.data, request.correlation_id)
    if result.get("error") == "Timed out waiting for the worker":
        response.status_code = 504
    return result
```

Worker

```
# code/hw8/worker/consumer.py
def process_request(request: dict[str, Any]) -> dict[str, Any]:
    """Run one operation and return the response body (without correlation_id)."""
    operation = request.get("operation")
    data = request.get("data")

    if operation == "CREATE_LISTING":
        error = validate_listing(data)
        if error:
            return {"success": False, "error": error}
        listing_id = str(uuid.uuid4())  # the worker, not the client, generates the id
        listings[listing_id] = {
            "listing_id": listing_id,
            "propertyTitle": data["propertyTitle"].strip(),
            "propertyLocation": data["propertyLocation"].strip(),
            "submitterEmail": data["submitterEmail"].strip(),
            "propertyDescription": data["propertyDescription"].strip(),
            "propertyCategory": data["propertyCategory"],
            "termsAccepted": "true",
            "submissionDate": datetime.now(UTC).isoformat(timespec="seconds"),
        }
        return {"success": True, "listing_id": listing_id, "message": "Listing created"}

    if operation == "GET_LISTING":
        listing_id = data.get("listing_id") if isinstance(data, dict) else None
        if not isinstance(listing_id, str) or not listing_id:
            return {"success": False, "error": "listing_id is required"}
        if listing_id not in listings:
            return {"success": False, "error": "Listing not found"}
        return {"success": True, "listing": listings[listing_id]}

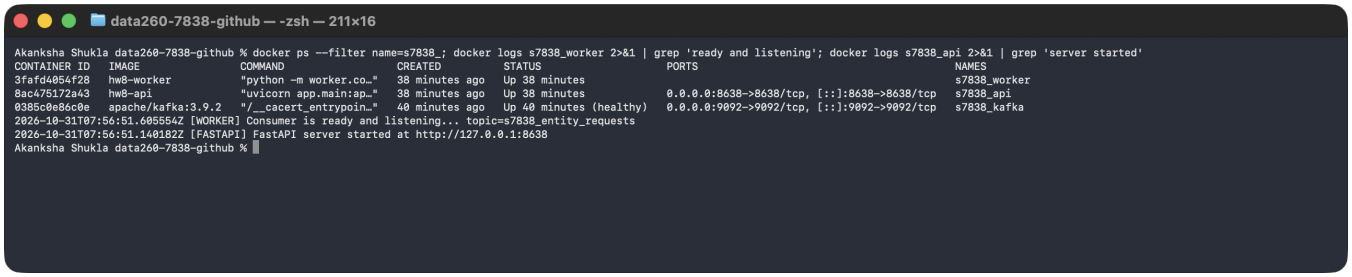
    return {"success": False, "error": "Unknown operation"}
```

```
# code/hw8/worker/consumer.py
        async for msg in consumer:
            try:
                request = json.loads(msg.value.decode())
                if not isinstance(request, dict):
                    raise ValueError("message must be a JSON object")
            except ValueError as exc:
                log("WORKER", f"skipped unreadable message offset={msg.offset}: {exc}")
                await consumer.commit({TopicPartition(msg.topic, msg.partition): msg.offset + 1})
```

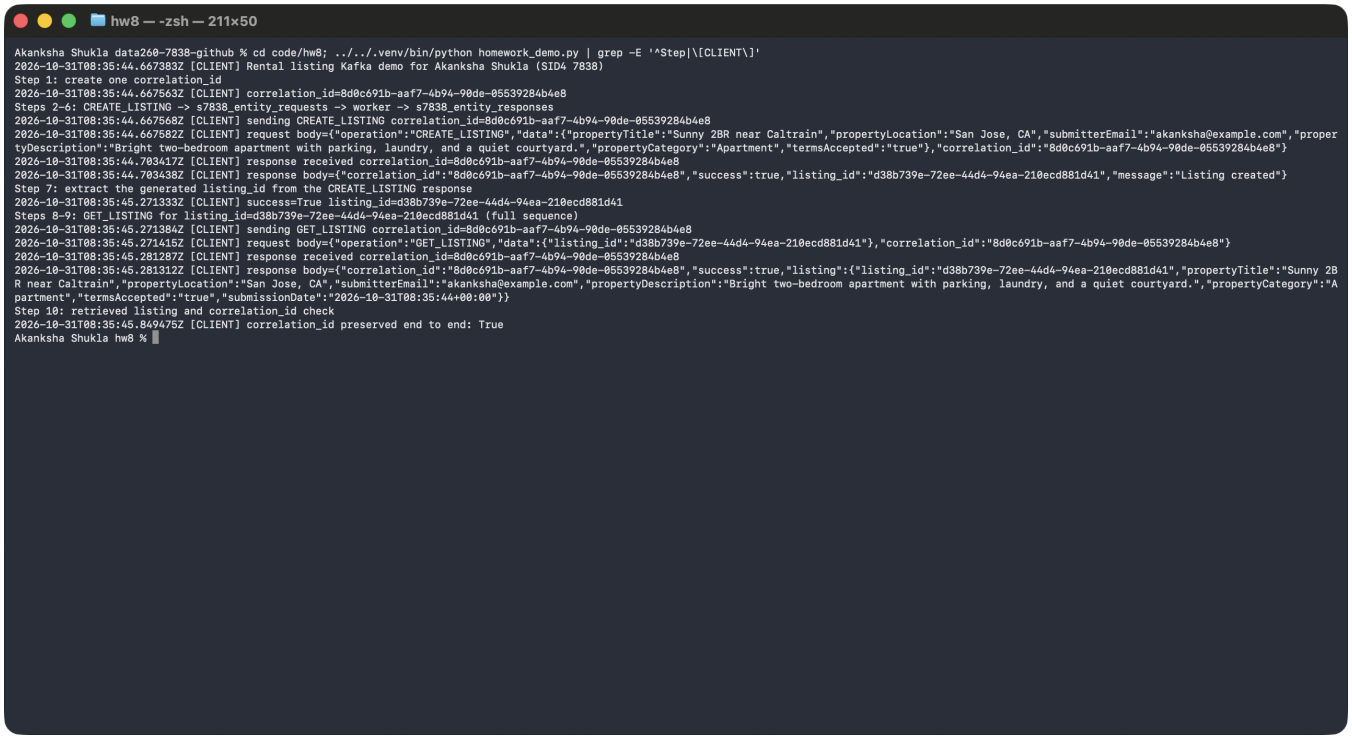
homework_demo.py

```
# code/hw8/homework_demo.py
def send(operation: str, data: dict, correlation_id: str) -> tuple[dict, list[str]]:
    """Send one request through the API and return (response, time-ordered log lines)."""
    since = utc_iso()
    request = {"operation": operation, "data": data, "correlation_id": correlation_id}
    own = [client_line(f"sending {operation} correlation_id={correlation_id}")]
    own.append(client_line(f"request body={json.dumps(request, separators=(',', ':'))}"))
    response = httpx.post(URL, json=request, timeout=30).json()
    own.append(client_line(f"response received correlation_id={response.get('correlation_id')}"))
    own.append(client_line(f"response body={json.dumps(response, separators=(',', ':'))}"))
    time.sleep(0.5)  # let the container logs flush
```

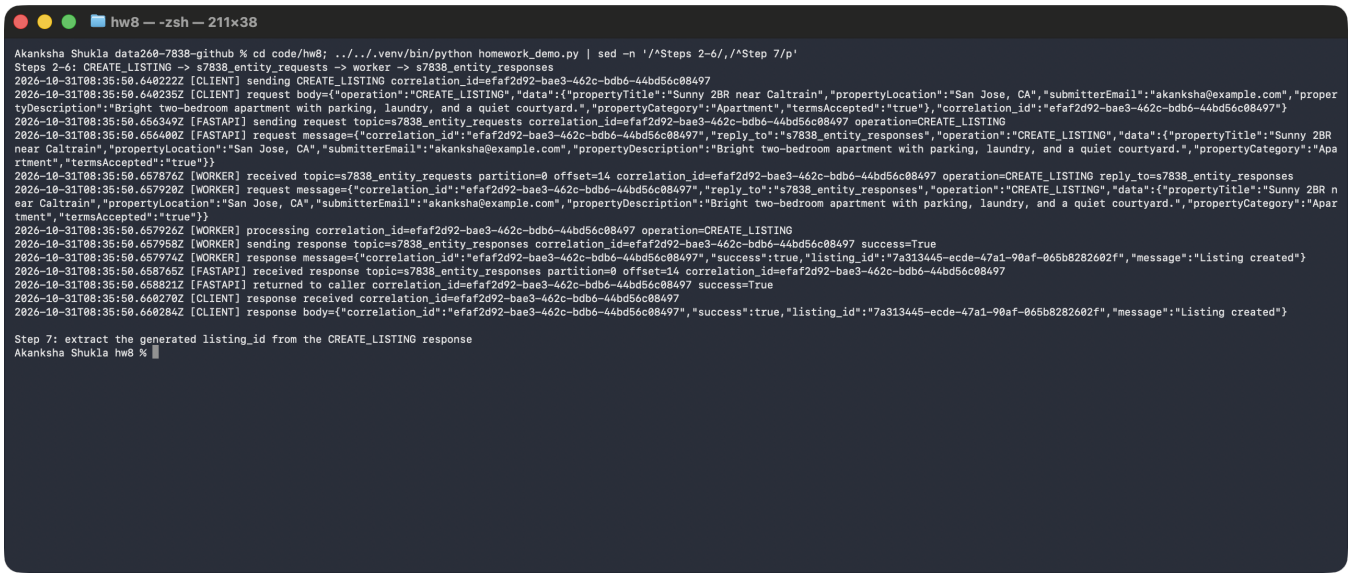
Screenshot - docker ps and the worker/API ready logs



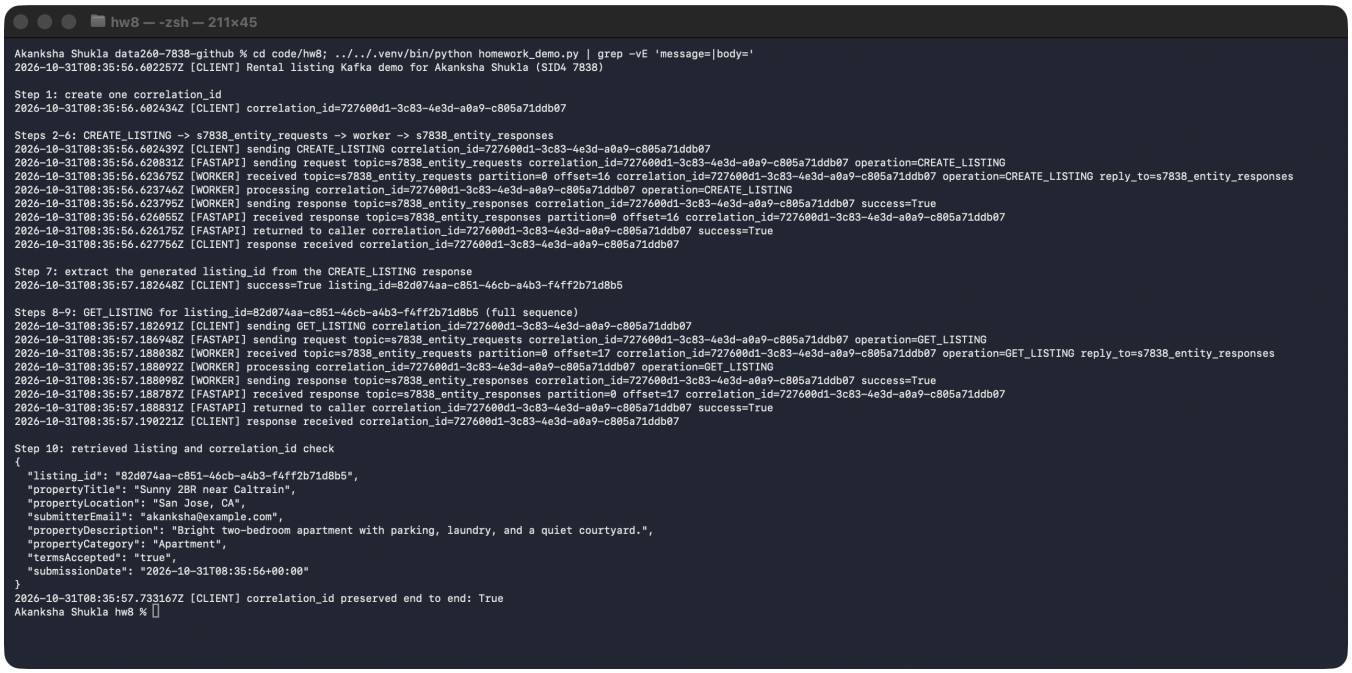
Screenshot - CREATE_LISTING and GET_LISTING execution



Screenshot - Kafka message flow for CREATE_LISTING



Screenshot - Full CREATE -> GET sequence in order



Part 2 - Three-agent Kafka pipeline

Sender	Reads topic	Sends message to
You (send_question.py)	-	s7838_inbox (a question)
Planner (planner.py)	s7838_inbox	s7838_tasks (a plan)
Writer (writer.py)	s7838_tasks	s7838_drafts (a draft answer)
Reviewer (reviewer.py)	s7838_drafts	s7838_final (an approval)

Planner, Writer, and Reviewer code

```
# code/hw8/planner.py
async def handle(message: dict[str, Any]) -> dict[str, Any]:
```

```
    question = message.get("question")
    if not isinstance(question, str) or not question.strip():
        raise ValueError("inbox message needs a non-empty question")
    steps = await plan_steps(question)
    if len(steps) < 2:
        raise ValueError(f"model returned fewer than two plan steps: {steps}")
    result = {
        "correlation_id": message["correlation_id"],  # preserved unchanged
        "question": question,
        "plan": steps[:3],
    }
    if message.get("entity_id"):
        result["entity_id"] = message["entity_id"]
    return result
```

```
# code/hw8/writer.py
async def handle(message: dict[str, Any]) -> dict[str, Any]:
    question, plan = message.get("question"), message.get("plan")
    if not isinstance(question, str) or not question.strip():
        raise ValueError("task message needs a non-empty question")
    if not isinstance(plan, list) or not plan:
        raise ValueError("task message needs a non-empty plan list")
    # work_ms lets the failure tests make processing take a known amount of time.
    await asyncio.sleep(float(message.get("work_ms", 0)) / 1000)
    result = {
        "correlation_id": message["correlation_id"],  # preserved unchanged
        "question": question,
        "plan": plan,
        "draft": await write_draft(question, plan),
    }
    for field in ("entity_id", "seq"):
        if field in message:
            result[field] = message[field]
    return result
```

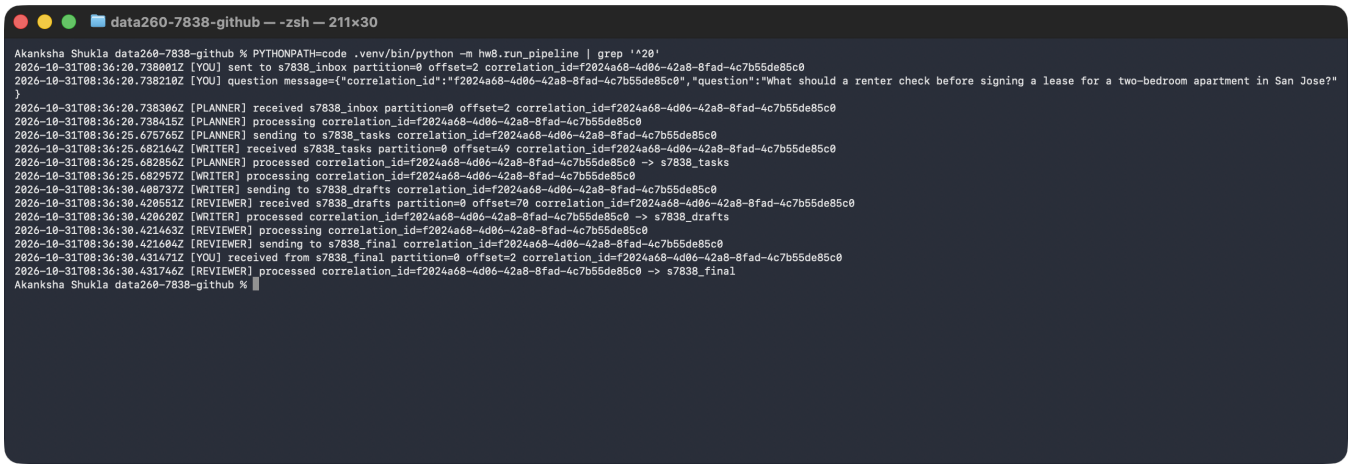
```
# code/hw8/reviewer.py
async def handle(message: dict[str, Any]) -> dict[str, Any]:
    draft = message.get("draft")
    if not isinstance(draft, str):
        raise ValueError("draft message needs a draft string")
    # Objective review rule: a usable answer is non-empty prose with at least MIN_WORDS words.
    approved = len(draft.split()) >= MIN_WORDS
    return {
        "correlation_id": message["correlation_id"],  # preserved unchanged
        "status": "approved" if approved else "rejected",
        "answer": draft,
    }
```

The agents are separate processes started with python planner.py, python writer.py, python reviewer.py, and python send_question.py. Planner and Writer call qwen2:7b (temperature 0, seed 7838); the Reviewer applies an objective rule (at least 15 words).

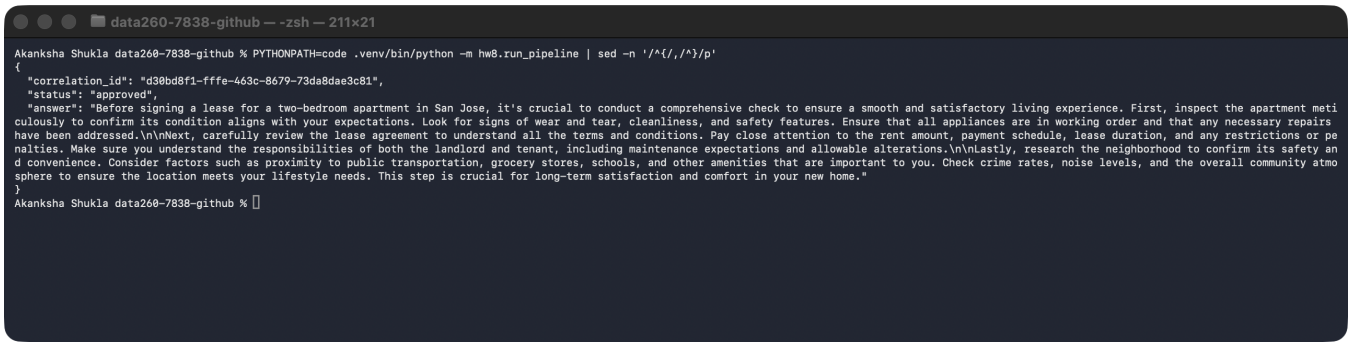
Screenshot - Topics created with the Kafka CLI (tasks has 2 partitions)



Screenshot - One complete message journey, inbox to final



Screenshot - Final JSON message with status and answer



How Kafka helped the agents coordinate

- Each agent only knows its input and output topic, so no agent calls another directly.
- Messages wait in the topic while an agent is busy or restarting, so the producer never blocks on a slow model call.
- The correlation_id travels in every message, so the final answer can be tied to the original question.
- The live run used qwen2:7b and finished in 6.6 s with status approved.

Part 3 - Failure behavior

Failure table with before/after evidence

Failure scenario	Before fix	After fix
Duplicate delivery	Same correlation_id delivered twice: processed=2, drafts published=2 (extra side effect).	Idempotency store: processed=1, duplicates skipped=1, drafts published=1.
Consumer killed mid-work	Offset committed on receipt: after SIGKILL committed offset=6 (message at 5); restarted Writer never saw it; drafts published=0 - message lost.	Offset committed after processing: committed offset after kill=6; restarted Writer received offset 6 again, finished it, and committed 7; drafts published=1.
Poison message	Unlimited retries: 54 failed attempts in 12 s, never dead-lettered; the next message was blocked (processed=False).	3 attempts, then 1 record in s7838_dead_letter (count=1); the next message was processed (processed=True).
Two partitions / two consumers	No key: related messages split across both partitions; entities processed out of order: listing-1.	Key = entity_id: every entity stayed in one partition and in order (all_in_order=True, all_in_one_partition=True).

Before/after runs differ only in the setting for that row (IDEMPOTENT, COMMIT_MODE, RETRY_MODE, or the message key). The Writer ran as a real process with an offline stand-in model so each run is quick and repeatable.

Idempotency, offset-commit, retry/dead-letter, and partition-key code

```
# code/hw8/reliability.py
class IdempotencyStore:
    """Durable set of processed correlation_ids (SQLite file, so restarts keep it)."""

    def __init__(self, path: str | Path) -> None:
        Path(path).parent.mkdir(parents=True, exist_ok=True)
        self.db = sqlite3.connect(path)
        self.db.execute("CREATE TABLE IF NOT EXISTS processed (correlation_id TEXT PRIMARY KEY)")
        self.db.commit()

    def seen(self, correlation_id: str) -> bool:
        row = self.db.execute(
            "SELECT 1 FROM processed WHERE correlation_id = ?", (correlation_id,)
        ).fetchone()
        return row is not None

    def mark(self, correlation_id: str) -> None:
        self.db.execute("INSERT OR IGNORE INTO processed VALUES (?)", (correlation_id,))
        self.db.commit()
```

```
# code/hw8/reliability.py
async def commit_offset(consumer: AIOKafkaConsumer, msg: Any) -> None:
    """Commit "next message to read" for this message's partition."""
    await consumer.commit({TopicPartition(msg.topic, msg.partition): msg.offset + 1})
```

```
# code/hw8/reliability.py
async def run_with_retries(
    attempt_once: Callable[[], Awaitable[Any]],
    retry_mode: str,
    max_attempts: int,
    emit: Callable[..., None],
    backoff: float = 0.3,
) -> tuple[bool, Any, int]:
    """Return (ok, output_or_error, attempts).

    bounded:   stop after max_attempts so the caller can dead-letter the message.
    unbounded: never gives up (the "before" behavior that blocks the partition).
    """
    attempt = 0
    while True:
        attempt += 1
        try:
```

```
            return True, await attempt_once(), attempt
        except Exception as exc:  # noqa: BLE001 - any failure counts as a failed attempt
            will_retry = retry_mode != "bounded" or attempt < max_attempts
            emit("attempt_failed", attempt=attempt, error=repr(exc)[:160], will_retry=will_retry)
            if not will_retry:
                return False, repr(exc)[:160], attempt
            await asyncio.sleep(backoff)
```

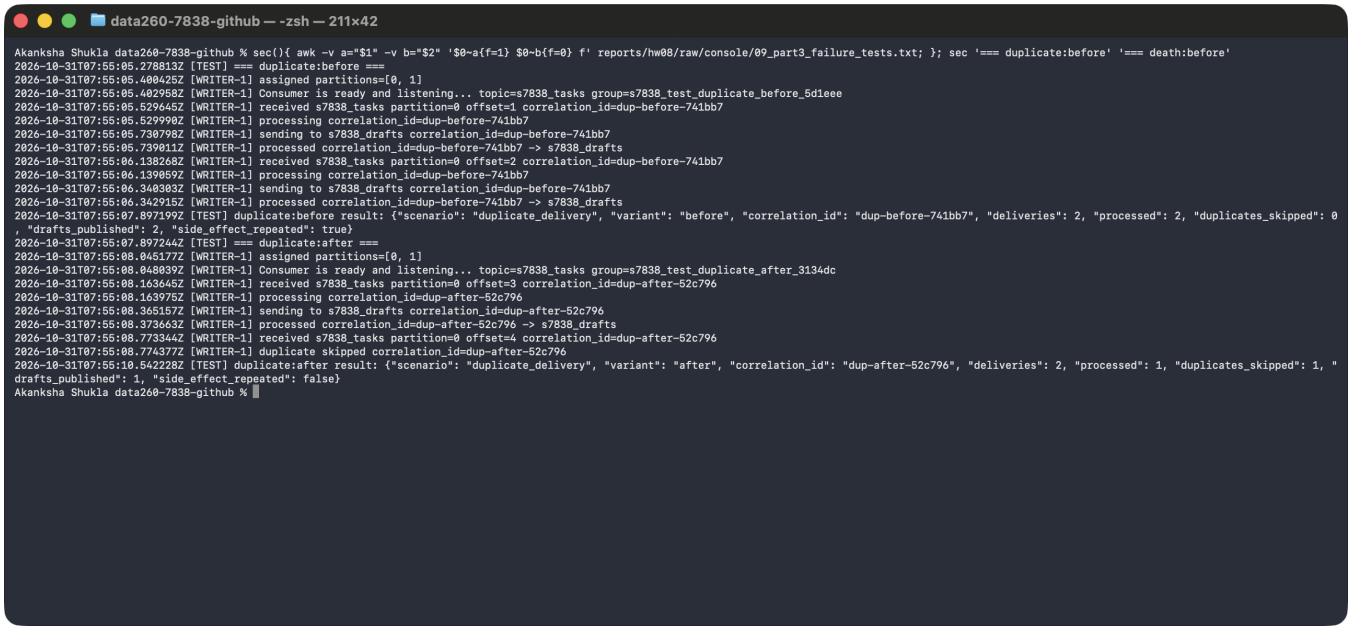
```
# code/hw8/reliability.py
def partition_key(message: dict[str, Any]) -> bytes:
    """Key by the entity the messages are about (falling back to the journey's correlation_id)."""
    return str(message.get("entity_id") or message["correlation_id"]).encode()
```

```
# code/hw8/agent_runtime.py
            if s.commit_mode == "early":  # "before": commit on receipt, work not done yet
                await commit_offset(consumer, msg)
                emit("committed", **detail)

            if store and correlation_id and store.seen(correlation_id):
                log(s.label, f"duplicate skipped correlation_id={correlation_id}")
                emit("duplicate_skipped", **detail)
    ...
            if s.commit_mode == "late":  # "after": commit only once the work is done
                await commit_offset(consumer, msg)
                emit("committed", **detail)
```

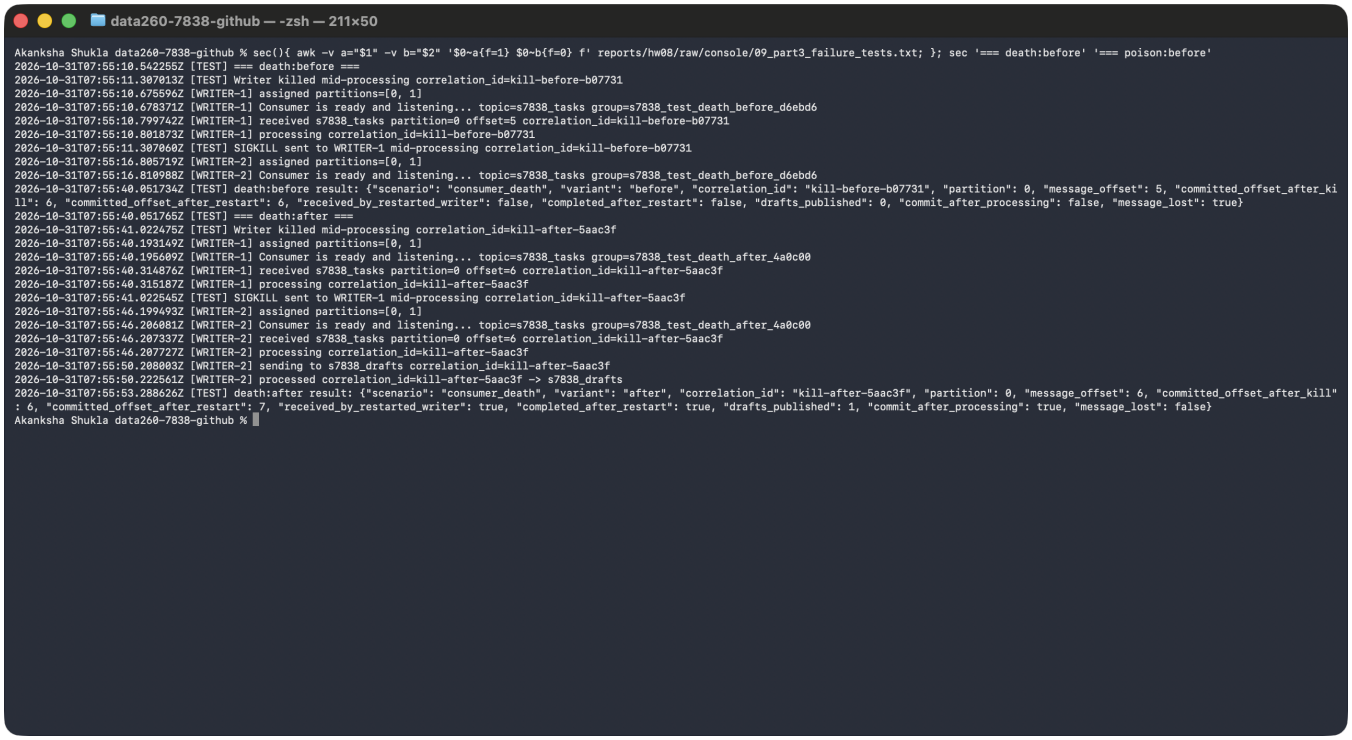
I. Duplicate delivery

Screenshot - Replay of the same correlation_id, before and after idempotency



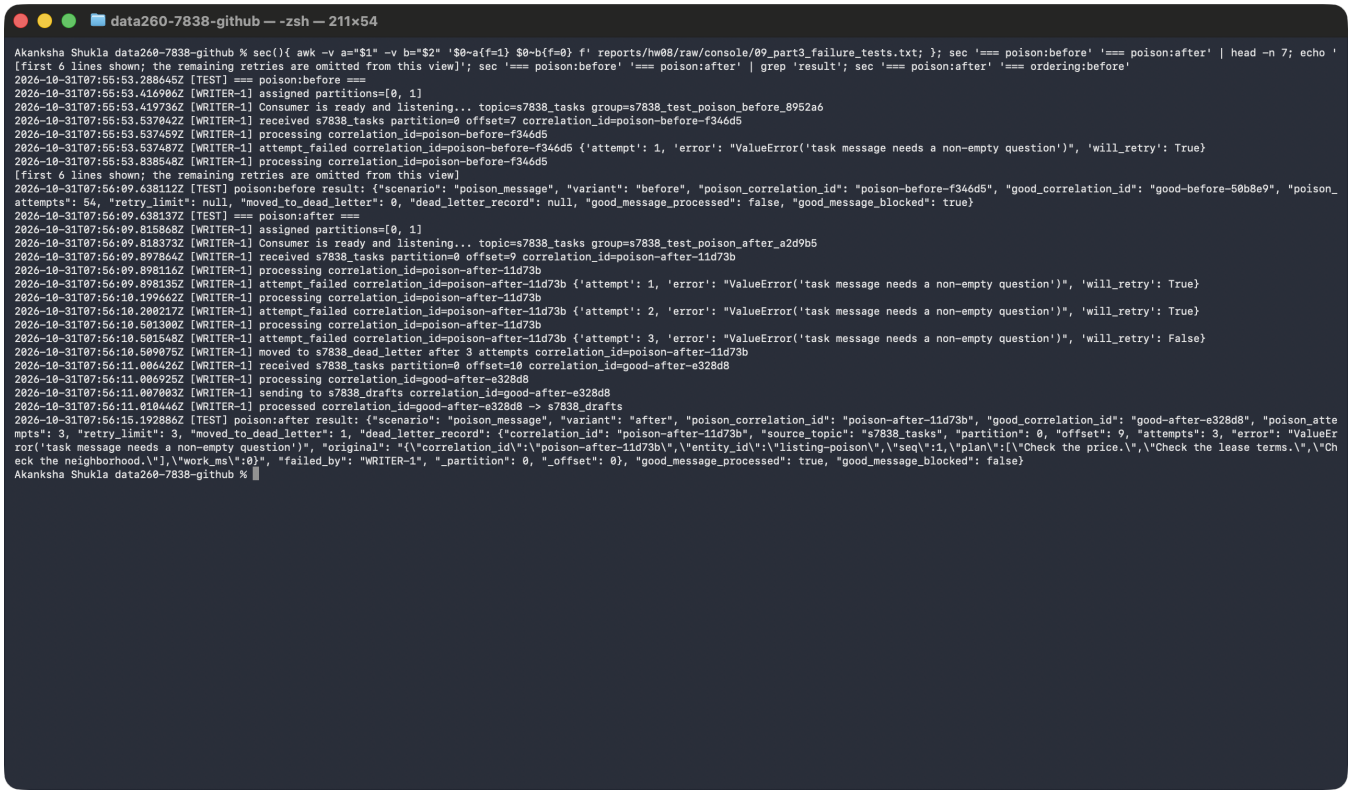
II. Consumer death

Screenshot - Writer killed mid-processing, then restarted



III. Poison message

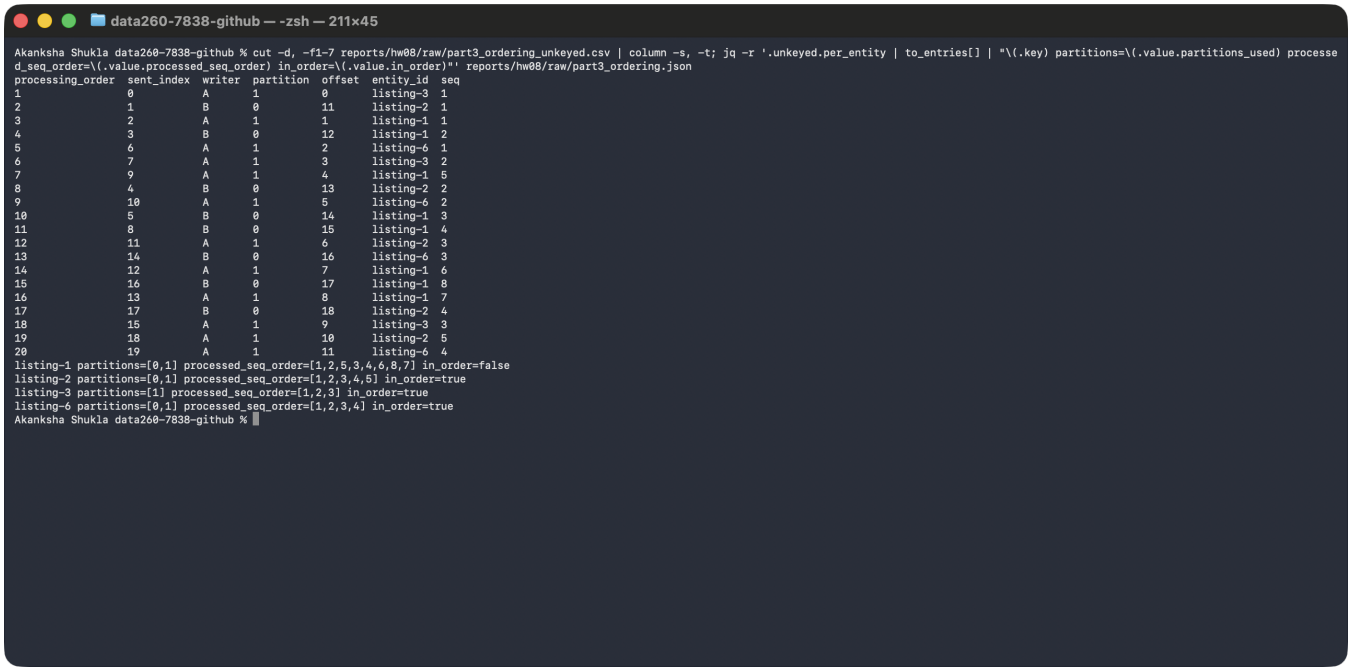
Screenshot - Poison message: unlimited retries versus bounded retries and dead letter



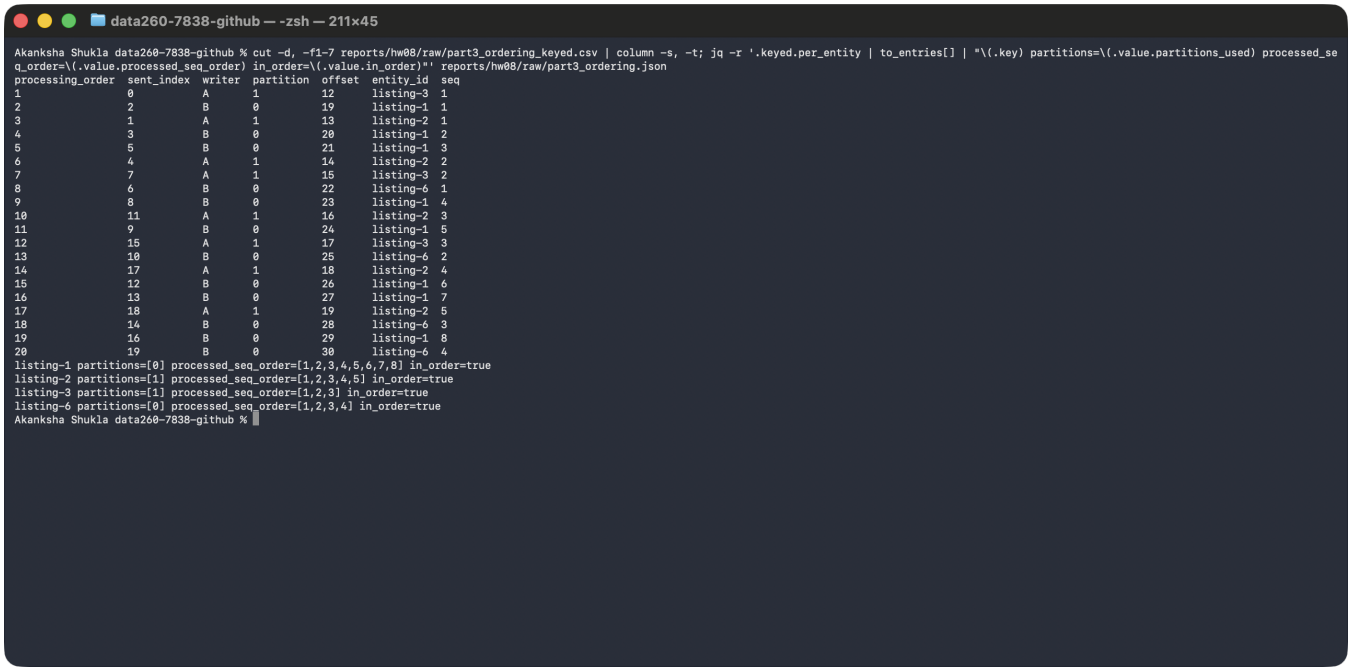
The 20 related messages are generated from VERIFY_SEED=267838 and sent through s7838_tasks (2 partitions) with two Writers in one consumer group. The raw rows are in raw/part3_ordering_unkeyed.csv and raw/part3_ordering_keyed.csv.

IV. Ordering

Screenshot - Without a key: related messages are processed out of order



Screenshot - With key = entity_id: related messages stay in one partition, in order



Comparison with the HW2 in-process graph

Two benefits gained by moving to Kafka

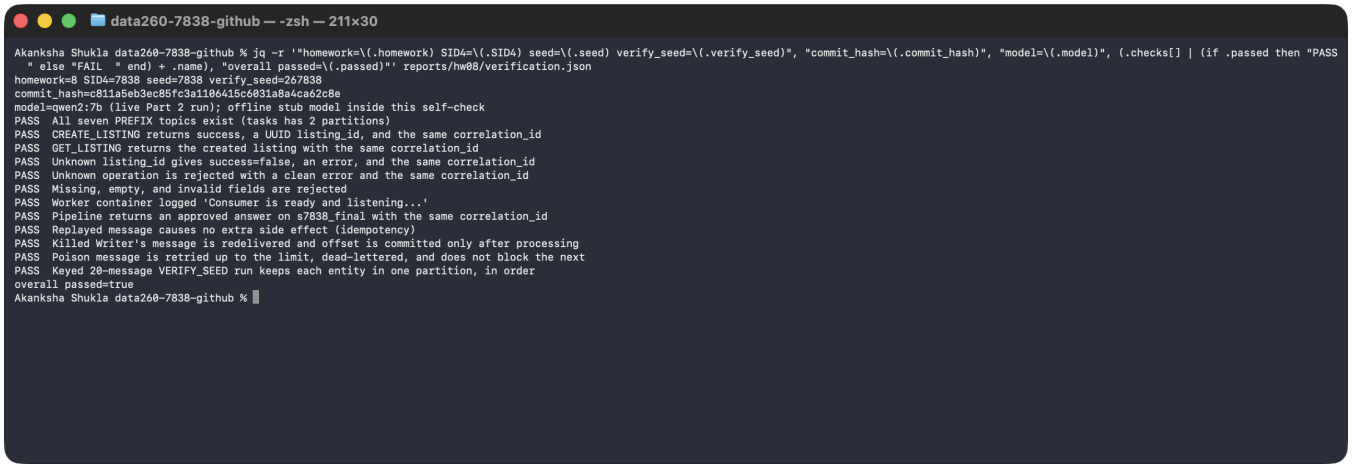
- Independent scaling: the Writer can run as several instances in one consumer group (two Writers shared s7838_tasks above) while Planner and Reviewer stay at one. The HW2 graph could only be scaled by copying the whole process.
- Decoupled failure: when the Writer was killed, its task stayed in Kafka and was finished after the restart. In the HW2 graph, one node calling the next directly meant a failed call failed the whole run with nothing saved to resume from.

Two responsibilities I now handle myself

- Duplicate delivery and bad messages: Kafka delivers at least once, so I added an idempotency store, a retry limit, and a dead-letter topic. A function call in HW2 ran exactly once per step.
- Ordering and offsets: ordering only holds within a partition, so I chose a key (entity_id), and I commit offsets only after processing. In HW2 the call order was the order of work.

Self-check result

Screenshot - make verify-hw08 wrote verification.json



AI use

What I used AI for, and what I did myself

I used Claude Code to read the HW8 requirements and to draft the Kafka worker, FastAPI wrapper, three agents, Docker Compose file, failure-test harness, self-check script, and report generator, following the structure of my earlier homework folders and the two example projects. I supplied the assignment, my fixed configuration (SID4 7838, port 8638, prefix s7838, seeds), and my Docker and Ollama environment, and I am responsible for the submitted results.

One AI output that was wrong, or one thing I verified

The first FastAPI wrapper created its Kafka producer and consumer when the module was imported, a pattern copied from the example project. With the installed aiokafka 0.15 this was wrong: the API container crashed at startup with "The object should be created within an async function" and kept restarting. A second problem appeared in the first demo run: merging client, API, and worker log lines by millisecond timestamp put the worker's "processing" line before its "received" line, which would have misdescribed the message flow.

How I detected the problem or verified the result

I found the crash by running `docker ps` (the API container showed a fresh restart each time) and reading `docker logs s7838_api`, and I found the log-order problem by reading the first `homework_demo.py` output against the expected order of the ten required steps. I then verified the fix with a real CREATE_LISTING followed by GET_LISTING through Kafka, and with the self-check script, which tests that the same correlation_id comes back on every response.

What I changed and why it works now

I moved client creation into `KafkaRPC.start()`, which runs inside the event loop, and I rebuilt the image. I switched log timestamps to microseconds and made the API and worker log "sending ..." before they publish, so each line shows the order in which things happened. The API now starts cleanly, logs `FastAPI server started at http://127.0.0.1:8638`, and the demo prints send, receive, process, reply, and client-response lines in the correct order.

Self-check commit: c811a5eb3ec85fc3a1106415c6031a8a4ca62c8e. Every screenshot is a capture of the Terminal app running the command shown (the full run is in reports/hw08/RUN_LOG.txt and raw/).